

Basics of ML - No train Size (Just to understand)

1: How Models Work:

कहानी: मान लीजिए आपको अपने किसी के लिए ऐसे मॉडल्स बनाने हैं जो पुराने घरों के डेटा को देखकर नए घरों की कीमत (Price Prediction) का अंदाज़ा लगा सकें।

मशीन लर्निंग (Machine Learning): यह भी बिल्कुल वैसे ही काम करती है जैसे हम पुराने अनुभवों या पैटर्न्स को देखकर भविष्य का अंदाज़ा लगाते हैं।

Decision Tree (डिसीजन ट्री): यह सबसे आसान मॉडल है जो डेटा को अलग-अलग सवाल/कंडीशंस (जैसे कमरों की संख्या, प्लॉट का साइज़) के आधार पर बांटता है और आखिर में कीमत बताता है।

Training / Fitting: पुराने डेटा का इस्तेमाल करके मॉडल को सिखाना कि पैटर्न क्या है, 'ट्रेनिंग' कहलाता है।

Leaf: ट्री का सबसे आखिरी पॉइंट जहां पर हमें फाइनल प्रेडिक्शन (कीमत) मिलती है।

2: Basic Data Exploration (डेटा को समझना)

Pandas लाइब्रेरी: मशीन लर्निंग में डेटा के साथ काम करने के लिए Python की सबसे मुख्य लाइब्रेरी Pandas का इस्तेमाल किया जाता है। इसे शॉर्ट में pd लिखते हैं।

DataFrame: यह एक्सेल शीट या SQL टेबल की तरह एक टेबल फॉर्मेट होता है जिसमें डेटा स्टोर होता है।

```
In [7]: import pandas as pd

# डेटा फाइल का रास्ता (Path) देकर उसे लोड करना
melbourne_file_path = 'melb_data.csv'
melbourne_data = pd.read_csv(melbourne_file_path)
```

```
In [8]: # डेटा के मुख्य आँकड़े (Summary Statistics) देखना
melbourne_data.describe()
```

Out[8]:

	Rooms	Price	Distance	Postcode	Bedroom2	Bathroom
count	13580.000000	1.358000e+04	13580.000000	13580.000000	13580.000000	13580.000000
mean	2.937997	1.075684e+06	10.137776	3105.301915	2.914728	1.534242
std	0.955748	6.393107e+05	5.868725	90.676964	0.965921	0.691712
min	1.000000	8.500000e+04	0.000000	3000.000000	0.000000	0.000000
25%	2.000000	6.500000e+05	6.100000	3044.000000	2.000000	1.000000
50%	3.000000	9.030000e+05	9.200000	3084.000000	3.000000	1.000000
75%	3.000000	1.330000e+06	13.000000	3148.000000	3.000000	2.000000
max	10.000000	9.000000e+06	48.100000	3977.000000	20.000000	8.000000



.describe() का मतलब: यह कमांड हर कॉलम का सांख्यिकीय विवरण (Statistical Overview) देती है:

count: कुल कितने घरों का डेटा उपलब्ध है (कितनी वैल्यूज़ मिसिंग नहीं हैं)।

mean: औसत (Average) कीमत या संख्या।

std: स्टैंडर्ड डेविएशन (डेटा कितना फैला हुआ है)।

min / max: सबसे कम और सबसे ज़्यादा वैल्यू।

3: Your First Machine Learning Model (आपका पहला मशीन लर्निंग मॉडल)

मॉडल बनाने के 4 मुख्य चरण (Steps):

Define: तय करना कि मॉडल किस प्रकार का होगा (जैसे- Decision Tree)।

Fit: पैटर्न सीखने के लिए डेटा (Training Data) का उपयोग करना।

Predict: नए डेटा के आधार पर भविष्यवाणियां करना।

Evaluate: देखना कि मॉडल कितना सही या गलत है।

```
In [10]: # dropna drops missing values (think of na as "not available")
melbourne_data = melbourne_data.dropna(axis=0)
```

```
In [12]: y = melbourne_data.Price
melbourne_features = ['Rooms', 'Bathroom', 'Landsize',
                      'Latitude', 'Longitude']

X = melbourne_data[melbourne_features]
```

```
In [14]: X.describe()
X.head()
```

```
Out[14]:
```

	Rooms	Bathroom	Landsize	Lattitude	Longitude
1	2	1.0	156.0	-37.8079	144.9934
2	3	2.0	134.0	-37.8093	144.9944
4	4	1.0	120.0	-37.8072	144.9941
6	3	2.0	245.0	-37.8024	144.9993
7	2	1.0	256.0	-37.8060	144.9954

```
In [15]: from sklearn.tree import DecisionTreeRegressor

# 1. Model Define करें
melbourne_model = DecisionTreeRegressor(random_state=1)

# 2. Model Fit करें (डेटा से सीखना)
melbourne_model.fit(X, y)

# 3. Predict करें (नए घरों की कीमतें निकालना)
print("Making predictions for the following 5 houses:")
print(X.head())
print("The predictions are")
print(melbourne_model.predict(X.head()))
```

Making predictions for the following 5 houses:

	Rooms	Bathroom	Landsize	Lattitude	Longitude
1	2	1.0	156.0	-37.8079	144.9934
2	3	2.0	134.0	-37.8093	144.9944
4	4	1.0	120.0	-37.8072	144.9941
6	3	2.0	245.0	-37.8024	144.9993
7	2	1.0	256.0	-37.8060	144.9954

The predictions are

[1035000. 1465000. 1600000. 1876000. 1636000.]

Features: जिन कॉलम या डेटा का इस्तेमाल करके हम प्रेडिक्शन करते हैं (जैसे- कमरों की संख्या, एरिया)।

Target: जिसकी हम भविष्यवाणी करना चाहते हैं (जैसे- घर की कीमत)।

Predictions are below:

```
In [16]: print("Making predictions for the following 5 houses:")
print(X.head())
print("The predictions are")
print(melbourne_model.predict(X.head()))
```

Making predictions for the following 5 houses:

	Rooms	Bathroom	Landsize	Lattitude	Longtitude
1	2	1.0	156.0	-37.8079	144.9934
2	3	2.0	134.0	-37.8093	144.9944
4	4	1.0	120.0	-37.8072	144.9941
6	3	2.0	245.0	-37.8024	144.9993
7	2	1.0	256.0	-37.8060	144.9954

The predictions are

[1035000. 1465000. 1600000. 1876000. 1636000.]



4: Model Validation(मॉडल का मूल्यांकन / जाँच करना)

मॉडल की क्वालिटी कैसे मापें? (Model Validation): मॉडल बनाने के बाद यह देखना ज़रूरी है कि वह कितना सही काम कर रहा है। इसके लिए हम MAE (Mean Absolute Error) का इस्तेमाल करते हैं।

MAE (Mean Absolute Error) क्या है?

यह हमें बताता है कि **मॉडल की भविष्यवाणी असली कीमत से औसतन (on average) कितनी दूर (गलत) है।**

उदाहरण: अगर असली कीमत USD 1,50,000 है और मॉडल ने USD 1,00,000 प्रेडिक्ट किया, तो एरर \$50,000 है। ऐसे ही सभी घरों के एरर का औसत **MAE** कहलाता है।

"In-Sample" स्कोर की समस्या (The Trap):

अगर हम उसी डेटा से मॉडल की जाँच करें जिससे उसे ट्रेन (सिखाया) किया गया था, तो मॉडल हमेशा सही रिजल्ट दिखाएगा क्योंकि वह उस डेटा को पहले ही रट चुका होता है। इसे "In-Sample" स्कोर कहते हैं, जो गलत तरीका है।

Validation Data (वैलिडेशन डेटा) का इस्तेमाल:

असली परफॉर्मेंस देखने के लिए हमें डेटा को दो हिस्सों में बांटना चाहिए—एक हिस्सा मॉडल को सिखाने के लिए (**Training Data**) और दूसरा हिस्सा मॉडल को **test** करने के लिए (**Validation Data**)।

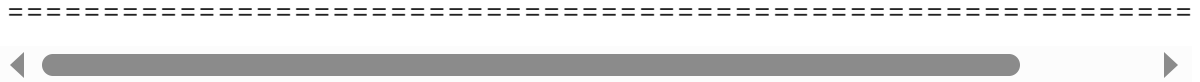
```
In [22]: from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from sklearn.tree import DecisionTreeRegressor

# 1. डेटा को ट्रेनिंग और वैलिडेशन हिस्सों में बांटना
train_X, val_X, train_y, val_y = train_test_split(X, y, random_state=0)

# 2. मॉडल डिफाइन और फिट करना (सिर्फ ट्रेनिंग डेटा पर)
melbourne_model = DecisionTreeRegressor()
melbourne_model.fit(train_X, train_y)
```

```
# 3. वैलिडेशन डेटा पर प्रेडिक्शन करना और MAE निकालना
val_predictions = melbourne_model.predict(val_X)
print(mean_absolute_error(val_y, val_predictions))
```

274413.89541639766



5. Underfitting and Overfitting

```
In [23]: from sklearn.metrics import mean_absolute_error
from sklearn.tree import DecisionTreeRegressor

def get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y):
    model = DecisionTreeRegressor(max_leaf_nodes=max_leaf_nodes, random_state=0)
    model.fit(train_X, train_y)
    preds_val = model.predict(val_X)
    mae = mean_absolute_error(val_y, preds_val)
    return(mae)
```

```
In [24]: # compare MAE with differing values of max_leaf_nodes
for max_leaf_nodes in [5, 50, 500, 5000]:
    my_mae = get_mae(max_leaf_nodes, train_X, val_X, train_y, val_y)
    print("Max leaf nodes: %d \t\t Mean Absolute Error: %d" %(max_leaf_nodes, my_
```

Max leaf nodes: 5	Mean Absolute Error: 385696
Max leaf nodes: 50	Mean Absolute Error: 279794
Max leaf nodes: 500	Mean Absolute Error: 261718
Max leaf nodes: 5000	Mean Absolute Error: 271320

1. Underfitting (बहुत कम Leaves होना) क्या होता है: अगर आप ट्री में बहुत कम लीव्स सेट करते हैं (जैसे सिर्फ 5 या 10 leaves), तो मॉडल बहुत साधारण (Simple) रह जाता है।

असर: यह डेटा के असली पैटर्न को पकड़ ही नहीं पाता, इसलिए ट्रेनिंग और वैलिडेशन—दोनों डेटा पर गलतियां (High Error) करता है।

2. Overfitting (बहुत ज़्यादा Leaves होना) क्या होता है: अगर आप ट्री को बहुत बड़ा बना देते हैं और हज़ारों leaves (जैसे 2000+ leaves) अनुमति देते हैं, तो ट्री हर एक घर की छोटी-छोटी बारीकी (Noise) को अलग से रट लेता है।

असर: यह ट्रेनिंग डेटा पर तो एकदम सही दिखेगा, लेकिन नए (Validation) डेटा पर बुरी तरह फेल हो जाएगा।

3. सही संतुलन (The Sweet Spot) कैसा होना चाहिए: हमें पत्तियों की संख्या (जैसे 50, 100, या 500) ऐसी रखनी चाहिए जिस पर Validation Error सबसे कम आए। न तो ट्री इतना छोटा हो कि सीख न पाए, और न इतना बड़ा हो कि सिर्फ रट ले।



आइए इसे आसान हिंदी में शॉर्ट में समझते हैं:

1. Random Forest क्या है? समस्या: एक अकेला Decision Tree या तो बहुत साधारण (Underfitting) हो सकता है या फिर चीज़ों को बहुत ज़्यादा रट सकता है (Overfitting)।

समाधान: Random Forest बहुत सारे Decision Trees को मिलाकर बनाया गया एक मॉडल है (इसे 'जंगलों' यानी कई पेड़ों का समूह कहा जाता है)।

2. यह कैसे काम करता है? यह अलग-अलग पेड़ों (Trees) से अलग-अलग प्रेडिक्शन करवाता है।

अंत में, सभी पेड़ों के प्रेडिक्शन का औसत (Average) निकालकर फाइनल आंसर देता है।

चूंकि इसमें बहुत सारे पेड़ मिलकर फैसला लेते हैं, इसलिए यह अकेले डिसीजन ट्री के मुकाबले कहीं ज़्यादा सटीक और भरोसेमंद होता है।

```
In [26]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

# 1. Random Forest मॉडल डिफाइन करें
forest_model = RandomForestRegressor(random_state=1)

# 2. मॉडल को फिट (Train) करें
forest_model.fit(train_X, train_y)

# 3. प्रेडिक्शन करें और MAE देखें
melbourne_preds = forest_model.predict(val_X)
print(mean_absolute_error(val_y, melbourne_preds))
```

207190.6873773146

फायदा: बिना किसी खास ट्यूनिंग के भी **Random Forest**, सिंगल डिसीजन ट्री के मुकाबले वैलिडेशन एरर को बहुत कम कर देता है।